JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Interface ScheduledExecutorService

All Superinterfaces:

`Executor`, `ExecutorService`

All Known Implementing Classes:

`ScheduledThreadPoolExecutor`

```
public interface ScheduledExecutorService
extends ExecutorService
```

An `ExecutorService` that can schedule commands to run after a given delay, or to execute periodically.

The `schedule` methods create tasks with various delays and return a task object that can be used to cancel or check execution. The `scheduleAtFixedRate` and `scheduleWithFixedDelay` methods create and execute tasks that run periodically until cancelled.

Commands submitted using the `Executor.execute(Runnable)` and `ExecutorService` `submit` methods are scheduled with a requested delay of zero. Zero and negative delays (but not periods) are also allowed in `schedule` methods, and are treated as requests for immediate execution.

All `schedule` methods accept *relative* delays and periods as arguments, not absolute times or dates. It is a simple matter to transform an absolute time represented as a `Date` to the required form. For example, to schedule at a certain future `date`, you can use: `schedule(task, date.getTime() - System.currentTimeMillis(), TimeUnit.MILLISECONDS)`. Beware however that expiration of a relative delay need not coincide with the current `Date` at which the task is enabled due to network time synchronization protocols, clock drift, or other factors.

The Executors class provides convenient factory methods for the ScheduledExecutorService implementations provided in this package.

Usage Example

Here is a class with a method that sets up a ScheduledExecutorService to beep every ten seconds for an hour:

```
import static java.util.concurrent.TimeUnit.*;
class BeeperControl {
  private final ScheduledExecutorService scheduler =
    Executors.newScheduledThreadPool(1);

  public void beepForAnHour() {
    final Runnable beeper = new Runnable() {
      public void run() { System.out.println("beep"); }
    };
    final ScheduledFuture<?> beeperHandle =
      scheduler.scheduleAtFixedRate(beeper, 10, 10, SECONDS);
    scheduler.schedule(new Runnable() {
      public void run() { beeperHandle.cancel(true); }
    }, 60 * 60, SECONDS);
  }
}
```

Since:

1.5

Method Summary

Abstract Methods

Modifier and Type	Method	Description
<V> ScheduledFuture<V>	schedule(Callable<V> callable, long delay, TimeUnit unit)	Creates and execu a ScheduledFutur that becomes enal after the given del
ScheduledFuture<?>	schedule(Runnable command, long delay, TimeUnit unit)	Creates and execu a one-shot action t becomes enabled the given delay.
ScheduledFuture<?>	scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit)	Creates and execu a periodic action t becomes enabled after the given ini delay, and

		subsequently with given period; that executions will commence after `initialDelay` the `initialDelay+pe` then `initialDela` `2 * period`, and s on.
`ScheduledFuture<?>`	`scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)`	Creates and execu a periodic action t becomes enabled after the given ini delay, and subsequently with given delay betwe the termination of execution and the commencement of next.

Methods inherited from interface java.util.concurrent.ExecutorService

`awaitTermination`, `invokeAll`, `invokeAll`, `invokeAny`, `invokeAny`, `isShutdown`, `isTerminated`, `shutdown`, `shutdownNow`, `submit`, `submit`, `submit`

Methods inherited from interface java.util.concurrent.Executor

`execute`

Method Detail

schedule

```
ScheduledFuture<?> schedule(Runnable command,
                            long delay,
                            TimeUnit unit)
```

Creates and executes a one-shot action that becomes enabled after the given delay.

Parameters:

command - the task to execute

delay - the time from now to delay execution

unit - the time unit of the delay parameter

Returns:

a ScheduledFuture representing pending completion of the task and whose get() method will return null upon completion

Throws:

RejectedExecutionException - if the task cannot be scheduled for execution

NullPointerException - if command is null

schedule

```
<V> ScheduledFuture<V> schedule(Callable<V> callable,
                                long delay,
                                TimeUnit unit)
```

Creates and executes a ScheduledFuture that becomes enabled after the given delay.

Type Parameters:

V - the type of the callable's result

Parameters:

callable - the function to execute

delay - the time from now to delay execution

unit - the time unit of the delay parameter

Returns:

a ScheduledFuture that can be used to extract result or cancel

Throws:

RejectedExecutionException - if the task cannot be scheduled for execution

NullPointerException - if callable is null

scheduleAtFixedRate

```
ScheduledFuture<?> scheduleAtFixedRate(Runnable command,
                                       long initialDelay,
                                       long period,
                                       TimeUnit unit)
```

Creates and executes a periodic action that becomes enabled first after the given initial delay, and subsequently with the given period; that is executions will commence after `initialDelay` then `initialDelay+period`, then `initialDelay + 2 * period`, and so on. If any execution of the task encounters an exception, subsequent executions are suppressed. Otherwise, the task will only terminate via cancellation or termination of the executor. If any execution of this task takes longer than its period, then subsequent executions may start late, but will not concurrently execute.

Parameters:

`command - the task to execute`

`initialDelay - the time to delay first execution`

`period - the period between successive executions`

`unit - the time unit of the initialDelay and period parameters`

Returns:

`a ScheduledFuture representing pending completion of the task, and whose get() method will throw an exception upon cancellation`

Throws:

`RejectedExecutionException - if the task cannot be scheduled for execution`

`NullPointerException - if command is null`

`IllegalArgumentException - if period less than or equal to zero`

scheduleWithFixedDelay

```
ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                          long initialDelay,
                                          long delay,
                                          TimeUnit unit)
```

Creates and executes a periodic action that becomes enabled first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next. If any execution of the task encounters an exception, subsequent executions are suppressed. Otherwise, the task will only terminate via cancellation or termination of the executor.

Parameters:

`command - the task to execute`

`initialDelay - the time to delay first execution`

`delay - the delay between the termination of one execution and the commencement of the next`

`unit - the time unit of the initialDelay and delay parameters`

Returns:

a ScheduledFuture representing pending completion of the task, and whose get() method will throw an exception upon cancellation

Throws:

RejectedExecutionException - if the task cannot be scheduled for execution

NullPointerException - if command is null

IllegalArgumentException - if delay less than or equal to zero

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.